Guia de Estudo e Treinamento

PC.8 — Chatbot de FAQ Escolar com Bag of Words, TF-IDF,
Similaridade de Cosseno e Votação por Maioria

Disciplina: Processamento de Linguagem Natural

Grupo cea.03.preprocessing_and_representation — g.4
Documento de preparação para apresentação oral (qualquer integrante pode ser sorteado)
Baseado em: pln_mathematical_text_representation.pdf (Prof. Fabrício Galende Marques de Carvalho)

1. Por que este documento existe

O professor disse que vai **sortear um integrante do grupo** para rodar, apresentar e explicar o projeto. Isso significa que **os três** precisam ser capazes de:

- Explicar, com as próprias palavras, o que é bag of words, TF-IDF, similaridade de cosseno e votação por maioria;
- Rodar o projeto do zero numa máquina limpa (ver `README.md`);
- Justificar as decisões de projeto (por que SQLite, por que este conjunto de categorias, por que este limiar de similaridade etc.);
- Responder perguntas "de pegadinha" sobre os limites da abordagem.

> **Como estudar com este guia:** leiam a Seção 2 (teoria) e a Seção 3 (como o código implementa a teoria) juntos, depois cada um treina sozinho respondendo em voz alta as perguntas da Seção 5 sem olhar a resposta. Por fim, façam juntos o roteiro de demonstração da Seção 6 pelo menos uma vez, ao vivo, no computador que será usado na apresentação.

2. O que o enunciado (PC.8) pede

Transcrição do enunciado, do documento *pln_mathematical_text_representation.pdf*, seção 7:

> "Utilizando a representação *bag of words* com transformação TF-IDF, desenvolva um chatbot que responda algumas questões frequentemente perguntadas em um contexto de uma escola (...). Defina claramente o contexto do seu chatbot (i.e., quais categorias de perguntas podem ser respondidas) e utilize a métrica de cosseno e votação por maioria para definir a resposta do seu chat. Desenvolva uma interface web simples e efetue todas as etapas de pré-processamento aplicáveis ao seu sistema. Utilize persistência de dados e não se esqueça de utilizar as etapas de pré-processamento estudadas ao longo do curso."

Checklist do que foi entregue, item por item do enunciado:

Exigência do enunciado	Onde está implementada
Representação bag of words com TF-IDF	`chatbot/engine.py` — `TfidfVectorizer` do scikit-learn
Contexto do chatbot claramente definido	5 categorias em `data/faq_seed.json`, exibidas na barra lateral da interface
Métrica de cosseno	`cosine_similarity` (scikit-learn) em `engine.py`

Votação por maioria	Função `answer()` em `engine.py`, contagem de votos entre o top-K
Interface web simples	Flask + `templates/index.html` (página única, estilo chat)
Pré-processamento completo	`chatbot/preprocessing.py`
Persistência de dados	SQLite (`chatbot/database.py`) — FAQ e histórico de conversas

3. Contexto (escopo) do chatbot

Um chatbot baseado em BOW/TF-IDF só reconhece o que está no seu **corpus de referência**. Por isso, definir o escopo é obrigatório e também serve de "contrato" do que ele promete responder. Este chatbot responde perguntas de 5 categorias, com 3 intenções (respostas fixas) cada, totalizando 15 intenções e ~73 frases de treino (paráfrases):

Categoria	Intenções cobertas
Horários e Calendário Acadêmico	horário das aulas, horário das disciplinas, calendário acadêmico/férias
Matrícula e Documentos	como se matricular, declaração de matrícula, trancamento
Biblioteca	horário de funcionamento, empréstimo de livro, renovação de empréstimo
Secretaria e Atendimento	telefone/e-mail, localização, horário de atendimento
Notas, Faltas e Avaliações	consulta de notas, consulta de faltas, regras de recuperação

Perguntas fora dessas 5 categorias (ex.: "qual a previsão do tempo?") caem no **fallback**: o chatbot diz que não entendeu e lista as categorias que sabe tratar — nunca "inventa" uma resposta.

4. Conceitos teóricos (o que o professor vai querer ouvir)

4.1 Engenharia de características (feature engineering)

Algoritmos de IA só operam sobre números. Texto é desestruturado e não numérico. A **engenharia de características** é o processo que converte um conjunto de palavras pré-processadas em uma representação numérica (vetor ou matriz), a partir de um corpus de referência. É exatamente isso que o pipeline `preprocessing.py → engine.py` faz.

4.2 Bag of Words (saco de palavras)

Representa um documento como um vetor onde cada posição corresponde a um termo do vocabulário do corpus de referência, **ignorando a ordem** das palavras. Se o corpus tem N termos distintos, cada documento vira um vetor em um espaço N-dimensional. Existem duas variantes:

- **BOW binário**: 1 se a palavra ocorre no documento, 0 caso contrário.
- **BOW por contagem (count vectorization)**: guarda quantas vezes cada palavra ocorre.

Exemplo do material do professor: corpus = "Eu gostei do produto. A qualidade é ótima!" → vocabulário D = {eu, gostei, do, produto, a, qualidade, é, ótima} (N=8). Para a frase "Ótima qualidade!", bow(s) = [0,0,0,0,0,1,0,1].

4.3 TF-IDF (Term Frequency – Inverse Document Frequency)

Problema do BOW puro: palavras muito frequentes (mas pouco informativas) dominam o vetor. O TF-IDF pondera cada termo por duas forças:

- **TF** (frequência do termo): quantas vezes o termo aparece *naquele* documento;
- **IDF** (frequência inversa nos documentos): penaliza termos que aparecem em muitos documentos do corpus (pouco discriminativos) e valoriza termos raros (mais discriminativos).

```
tfidf(w, d) = tf(w, d) · idf(w, D)        idf(w, D) = log( N / df(w, D) )
```

Com suavização (evita divisão por zero quando uma palavra nova não está no corpus de treino — exatamente a fórmula usada pelo `TfidfVectorizer(smooth_idf=True)` do scikit-learn, que é o que usamos no projeto):

```
tfidf(w, d) = [tf(w, d) + 1] · [1 + log( (N+1) / (1+df(w, D)) )]
```

Depois, o vetor é normalizado pela **norma L2** (norma euclidiana), porque o que importa é a *direção* do vetor, não sua magnitude: `tfidf_norm = tfidf / ||tfidf||`.

4.4 Similaridade de cosseno

Mede o ângulo entre dois vetores de características (não a distância "em linha reta"). Quanto mais próximo de 1, mais parecidos os vetores (e as frases); quanto mais próximo de 0, mais diferentes.

> `cos(θ) = (v1 · v2) / (‖v1‖ · ‖v2‖)`

Como os vetores TF-IDF já foram normalizados (norma L2 = 1), na prática `cos(θ) = v1 · v2` (produto escalar simples). É por isso que a similaridade de cosseno "combina" naturalmente com TF-IDF.

4.5 Votação por maioria (majority voting)

Em vez de responder sempre com base na *única* pergunta de referência mais parecida (o que seria frágil — uma única frase de treino mal formulada poderia desviar a resposta), o sistema:

1. calcula a similaridade de cosseno da pergunta do usuário contra **todas** as perguntas de referência;
2. pega as **top-K** mais similares (K = 5, configurável);
3. cada uma dessas K perguntas "vota" na intenção (resposta) a que pertence;
4. a intenção mais votada vence; empate é desfeito pela soma das similaridades dos votos e, por último, pela maior similaridade individual.

Isso é conceitualmente o mesmo princípio do classificador **K-Nearest Neighbors (KNN)** aplicado sobre o espaço vetorial TF-IDF, usando cosseno como métrica de distância.

4.6 Pipeline de pré-processamento

Aplicada a toda frase (tanto no corpus de referência quanto na pergunta do usuário), sempre na mesma ordem:

#	Etapa	Por quê
1	Minúsculas	"Horário" e "horário" devem virar o mesmo token
2	Remoção de acentos	"horário" → "horario": usuários digitam sem acento com frequência; sem isso o vocabulário "duplica" e a similaridade despenca (ver Seção 7)
3	Remoção de caracteres especiais/pontuação	"aula?" e "aula" devem virar o mesmo token
4	Tokenização	quebra a frase em palavras (tokens)
5	Remoção de stopwords	remove palavras de baixo conteúdo semântico (artigos, preposições, verbos auxiliares) que só adicionariam ruído ao vetor

6	Stemização (RSLP, via NLTK)	reduz variações morfológicas ao mesmo radical, ex.: "matricular", "matrícula", "matriculado" → mesmo radical

A lista de stopwords do português fica embutida diretamente no código (não depende de download), para o chatbot nunca falhar por falta de internet numa apresentação. Já o stemmer RSLP depende de um pacote de dados do NLTK baixado uma única vez (`setup_nltk.py`) e depois fica em cache local.

4.7 Stemização x lematização

Stemização corta sufixos de forma mecânica/heurística (pode gerar radicais que não são palavras reais); lematização usa conhecimento linguístico para achar a forma "de dicionário" (lema) da palavra. Usamos **stemização** (RSLP) porque é mais leve, não exige um modelo linguístico completo do português (como o spaCy exigiria) e é suficiente para o BOW aproximar frases com a mesma raiz.

5. Como o código implementa a teoria

5.1 Visão geral da arquitetura

Camada	Arquivo	Responsabilidade
Pré-processamento	`chatbot/preprocessing.py`	minúsculas → acentos → pontuação → tokens → stopwords → stem
Motor do chatbot	`chatbot/engine.py`	TF-IDF (scikit-learn), cosseno, votação por maioria, fallback
Persistência	`chatbot/database.py`	SQLite: FAQ (categorias/intenções/perguntas) + histórico de conversas
Web	`app.py` + `templates/` + `static/`	Flask: página única de chat + API JSON
Dados	`data/faq_seed.json`	conteúdo editável do FAQ (categorias, respostas, frases de treino)

5.2 Fluxo completo de uma pergunta (passo a passo)

1. Usuário digita a pergunta na interface web (`templates/index.html` + `static/script.js`), que faz um POST para `/api/chat` .
2. O Flask (`app.py`) chama `engine.answer(pergunta)` .
3. O `TfidfVectorizer` (já treinado/"fit" no corpus de referência, ao iniciar a aplicação) transforma a pergunta no mesmo espaço vetorial do corpus — usando o **mesmo** vocabulário e os **mesmos** pesos IDF aprendidos.
4. `cosine_similarity` calcula a similaridade entre o vetor da pergunta e cada um dos vetores de referência.
5. Pega-se o top-5 (K=5) mais similares; se o melhor deles tiver similaridade abaixo de 0,15 (limiar mínimo), o sistema assume que não reconhece a pergunta.
6. Votação por maioria entre o top-5: a intenção mais votada vence (empate → soma de similaridades → maior similaridade individual).
7. A resposta, a categoria, a confiança e a interação completa são gravadas na tabela `conversation_log` do SQLite (persistência) e devolvidas para a interface.

5.3 Exemplo real, com números reais do sistema rodando

Pergunta do usuário: **"Qual o horário da biblioteca?"**

Posição no top-5	Pergunta de referência	Intenção	Similaridade (cosseno)
1º	Qual o horário de atendimento da biblioteca?	horario_biblioteca	0,767
2º	Qual o horário de funcionamento da biblioteca?	horario_biblioteca	0,693
3º	Qual o horário das aulas?	horario_aulas	0,465
4º	Que horas a biblioteca abre?	horario_biblioteca	0,397
5º	Qual o horário de atendimento da secretaria?	horario_atendimento_secretaria	0,380

Votação: **horario_biblioteca** recebe 3 votos (1º, 2º, 4º), contra 1 voto de *horario_aulas* e 1 voto de *horario_atendimento_secretaria*. A intenção vencedora é **horario_biblioteca**, com confiança reportada 0,767 (maior similaridade entre os votos vencedores). Essa tabela (com destaque dos votos) pode ser vista ao vivo clicando em "Mostrar detalhes técnicos" na interface — **usem isso na demonstração**, é a prova visual de que a votação por maioria está funcionando, e não só "pegar o mais parecido".

6. Uma armadilha real que encontramos (ótimo exemplo para a apresentação)

> **O que aconteceu:** ao testar a frase fora de escopo "Quero pedir uma pizza", o chatbot inicialmente respondeu com a intenção *trancamento* (confiança 0,48) — um falso positivo claro.

Por quê isso aconteceu (e por que é um ótimo exemplo didático de uma limitação do TF-IDF): a única frase de treino da intenção "trancamento" que começa com "Quero" ("Quero trancar o curso, o que devo fazer?") faz da palavra "quero" um termo *raro* no corpus — ela aparece em pouquíssimas frases de treino. O IDF é maior justamente para termos raros, então o TF-IDF deu a "quero" um peso alto, mesmo essa palavra não tendo relação nenhuma com a intenção de trancamento — é só uma forma educada de começar uma frase.

Correção aplicada: verbos de desejo/pedido genéricos e pouco discriminativos ("quero", "queria", "preciso", "desejo" etc.) foram adicionados à lista de stopwords, assim como já havia sido feito com "gostaria". Depois da correção, a mesma frase cai corretamente no fallback.

> **Por que contar isso ao professor é uma boa ideia:** mostra que o grupo não só implementou o algoritmo, mas *testou*, encontrou um caso real de falha, entendeu a *causa raiz* matemática (peso IDF de termo raro) e corrigiu com uma decisão de engenharia (curadoria de stopwords) — exatamente o tipo de raciocínio crítico que costuma ser valorizado numa arguição.

7. Perguntas que o professor pode fazer (treinem respondendo em voz alta)

P: O que é bag of words e qual a diferença entre a versão binária e a de contagem?

R: Bag of words representa um documento como um vetor onde cada dimensão é um termo do vocabulário do corpus, ignorando a ordem das palavras. Na versão binária, cada posição é 0 ou 1 (a palavra ocorre ou não); na versão por contagem, a posição guarda quantas vezes a palavra aparece. O projeto usa, na prática, a versão com TF-IDF, que parte de uma contagem e depois pondera cada termo.

P: Por que usar TF-IDF em vez de BOW puro (contagem simples)?

R: Porque no BOW puro, palavras muito frequentes (mas pouco informativas) podem dominar o vetor e mascarar palavras realmente discriminativas. O TF-IDF reduz o peso de termos muito comuns no corpus (alto df → baixo idf) e aumenta o peso de termos raros/discriminativos (baixo df → alto idf).

P: O que é IDF e qual sua fórmula?

R: IDF (Inverse Document Frequency) é log(N / df(w, D)), onde N é o número de documentos do corpus e df(w, D) é em quantos documentos o termo w aparece. No projeto usamos a versão suavizada do scikit-learn, [1 + log((N+1)/(1+df))], que evita divisão por zero e reduz o efeito de termos raríssimos.

P: O que é similaridade de cosseno e por que ela (e não distância euclidiana)?

R: É o cosseno do ângulo entre dois vetores: cos(θ) = (v1·v2)/(‖v1‖‖v2‖). Para texto, o que importa é a *direção* do vetor (quais palavras e com que peso relativo), não o tamanho do documento. Dois documentos com o mesmo "assunto" mas tamanhos diferentes têm vetores na mesma direção, logo cosseno alto, mesmo que a distância euclidiana entre eles seja grande.

P: Como funciona exatamente a "votação por maioria" no seu chatbot?

R: Pegamos as K perguntas de referência mais similares (cosseno) à pergunta do usuário (K=5). Cada uma delas pertence a uma intenção (resposta). Contamos quantos votos cada intenção recebeu entre essas K; a mais votada vence. Em caso de empate, desempatamos pela soma das similaridades dos votos daquela intenção e, se ainda empatar, pela maior similaridade individual.

P: Por que não simplesmente responder com a pergunta de referência de maior similaridade (top-1)?

R: Porque uma única frase de treino pode ser ambígua ou mal formulada e desviar a resposta ("ruído" de uma única amostra). Olhando para várias (top-K) e deixando a maioria decidir, o sistema fica mais robusto — é o mesmo princípio usado no classificador KNN.

P: O que acontece se o usuário perguntar algo totalmente fora do escopo do chatbot?

R: Calculamos a similaridade contra todo o corpus de referência; se a maior similaridade encontrada ficar abaixo de um limiar mínimo (0,15 no projeto), o sistema não arrisca uma resposta errada — devolve uma mensagem de fallback listando as categorias que ele sabe tratar.

P: Quais etapas de pré-processamento vocês aplicaram e por quê cada uma?

R: Minúsculas (normaliza maiúsculas/minúsculas), remoção de acentos (usuário digita sem acento com frequência), remoção de pontuação/caracteres especiais, tokenização, remoção de stopwords (remove ruído de palavras pouco informativas) e stemização via RSLP (une variações morfológicas da mesma palavra num só radical).

P: Por que stemização (RSLP) e não lematização?

R: Stemização é uma heurística de corte de sufixo, mais simples e leve computacionalmente; lematização exigiria um modelo linguístico completo do português (ex.: spaCy com modelo pt_core_news) para encontrar a forma de dicionário da palavra. Para o BOW aproximar palavras da mesma família (matricular/matrícula/matriculado), a stemização já é suficiente.

P: Que tipo de persistência de dados vocês usaram e por quê?

R: SQLite: um banco relacional leve, guardado em um único arquivo (data/faq.db), sem precisar instalar/configurar um servidor de banco separado. Guardamos tanto o conteúdo do FAQ (categorias, intenções, respostas, frases de treino) quanto o histórico de toda conversa (pergunta, resposta, categoria, confiança, data/hora). Daria para trocar por PostgreSQL com poucas mudanças, já que o acesso ao banco está isolado em um único módulo (chatbot/database.py).

P: O que é o "vocabulário"/"corpus de referência" do vectorizer, de onde ele vem?

R: É o conjunto de todas as palavras distintas (após pré-processamento) que aparecem nas perguntas de referência cadastradas no FAQ. O TfidfVectorizer é "treinado" (fit) uma vez, ao iniciar a aplicação, sobre essas perguntas; depois disso, tanto o corpus quanto qualquer pergunta nova do usuário são projetados nesse mesmo espaço vetorial fixo.

P: O que acontece se eu quiser adicionar uma nova pergunta/categoria ao FAQ?

R: Basta editar data/faq_seed.json (adicionar a categoria/intenção/respostas/frases de treino) e rodar "python seed_db.py --force" para recarregar o banco. Não é preciso mexer em código.

P: Quais as limitações dessa abordagem?

R: (1) BOW/TF-IDF ignora a ordem das palavras e não entende sinônimos que nunca apareceram no corpus de treino (ex.: se "docente" nunca apareceu, o sistema não sabe que é sinônimo de "professor"); (2) o sistema só reconhece o que está dentro do corpus de referência cadastrado — é tão bom quanto a variedade de frases de treino cadastradas; (3) termos raros podem ganhar peso IDF desproporcional mesmo sem relação semântica real com a intenção (vimos um caso real disso na Seção 6); (4) não há entendimento de contexto de conversa (cada pergunta é tratada de forma independente, sem memória do que foi perguntado antes).

P: Como isso poderia evoluir para ficar mais robusto (ex.: usando o que foi visto depois, na Seção 6 do material teórico)?

R: Substituindo (ou combinando) a representação TF-IDF por embeddings (ex.: Word2Vec), que capturam similaridade semântica mesmo entre palavras que nunca apareceram juntas no corpus, representando a frase pela média dos embeddings de suas palavras — conforme discutido na Seção 6 do material teórico do professor.

8. Roteiro de demonstração ao vivo

Pratiquem esta sequência pelo menos uma vez, de ponta a ponta, no computador/rede que será usado na apresentação.

1. Abrir terminal na pasta `pc8_chatbot_faq_escola` e ativar o ambiente virtual (`.venv\Scripts\Activate.ps1` no Windows).
2. Rodar `python tests/test_chatbot.py` e mostrar a saída: 100% de acurácia nos casos de teste, prova de que o motor foi validado automaticamente.
3. Rodar `python app.py` e abrir `http://127.0.0.1:5000` no navegador.
4. Mostrar a barra lateral com as 5 categorias (contexto do chatbot) — ler em voz alta para o professor.
5. Fazer uma pergunta "fácil" (ex.: "Qual o horário da biblioteca?") e clicar em "Mostrar detalhes técnicos" para expor a tabela de similaridade/votação.
6. Fazer uma pergunta com erro de acentuação/digitação (ex.: "Como faco minha matricula") para mostrar que o pipeline de pré-processamento lida bem com isso.
7. Fazer uma pergunta fora do escopo (ex.: "Qual a previsão do tempo?") e mostrar o fallback com a lista de categorias.
8. Contar o caso real da Seção 6 (pizza/trancamento) como exemplo de que o grupo testou e entendeu uma limitação real do TF-IDF.
9. Opcional: consultar `http://127.0.0.1:5000/api/history` para mostrar o histórico persistido no SQLite.

9. Glossário rápido

Termo	Significado resumido
Corpus	Conjunto de documentos/frases usado como referência para montar o vocabulário
Vocabulário / léxico	Conjunto de todos os termos distintos do corpus, após pré-processamento
Token	Cada unidade (geralmente palavra) resultante da tokenização de um texto
Stopword	Palavra de baixo conteúdo semântico (artigos, preposições etc.), normalmente removida
Stemização	Corte heurístico de sufixos para reduzir palavras ao mesmo radical
TF (term frequency)	Quantas vezes um termo aparece em um documento específico
DF (document frequency)	Em quantos documentos do corpus um termo aparece
IDF (inverse document frequency)	Peso que cresce quanto mais raro o termo é no corpus
Norma L2	Normalização euclidiana de um vetor, usada para focar na direção, não na magnitude
Similaridade de cosseno	Medida de quão "alinhados" dois vetores estão (0 a 1 para vetores não-negativos)
Intenção (intent)	Categoria de pergunta com uma resposta fixa associada, treinada com várias paráfrases
Votação por maioria	Decisão tomada pela intenção mais votada entre os K vizinhos mais similares
Fallback	Resposta padrão usada quando nenhuma intenção atinge o limiar mínimo de similaridade

Documento de apoio ao estudo, gerado como parte da entrega do PC.8 (cea.03.preprocessing_and_representation, g.4), com base no material teórico "Representação Matemática de Texto Utilizada em PLN" do Prof. Fabrício Galende Marques de Carvalho.